Regression from Scratch

DAEN 300

Fall 2026



Learning objectives

- Fit the same regression line using different Python libraries.
- Explain the least-squares objective and matrix solution.
- Calculate fitted values, residuals, SSE, and R^2.
- Distinguish statistical inference from predictive modeling.
- Compare a numerical optimization solver w/ gradient descent.
- Explain how gradient descent estimates the same coefficients iteratively.

Regression: Five Approaches

Same regression problem,
different computational approaches and modeling tools.

Approach	**Library**	**Main Concept**
Analytical least squares	NumPy	Solve least squares using linear algebra
Statistical regression	statsmodels	Estimate coefficients and perform inference
Machine learning	scikit-learn	Fit models and generate predictions
Convex optimization	CVXPY	Define an objective and solve numerically
Gradient descent	PyTorch	Iteratively update coefficients using gradients

1. One dataset, one regression problem

Suppose x is machine operating time (hours) and y is output (hundreds of units).

x	1	2	3	4	5	6	7	8
y	2	3	5	4	6	8	7	9

$$\boxed{\widehat{y} = b_0 + b_1 x}$$

- b_0: intercept; b_1: change in fitted output per hour.
- Residual: $e_i = y_i - \widehat{y}_i$.
- All code slides use these eight observations.

Code: Setup and data

```
# NumPy: arrays and numerical least squares
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
y = np.array([2, 3, 5, 4, 6, 8, 7, 9], dtype=float)
```

2. Analytical least squares

Choose b_0, b_1 to minimize the squared residuals:

$$\boxed{\mathrm{SSE}(b_0, b_1) = \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i)^2}$$

In matrix notation, $\widehat{\mathbf{y}} = X\mathbf{b}$, where

$$X = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}.$$

If X has full column rank, the analytical OLS solution is

$$\boxed{\widehat{\mathbf{b}} = (X^T X)^{-1} X^T \mathbf{y}}.$$

Numerically, prefer a least-squares solver to computing the inverse.

Code: NumPy least squares

```
# First column of ones gives an intercept.
X = np.column_stack((np.ones(len(x)), x))

# Stable least-squares solver.
beta = np.linalg.lstsq(X, y, rcond=None)[0]

?np.linalg.lstsq # check documentation!!

intercept, slope = beta
print("Intercept:", intercept)
print("Slope:", slope)

# Predicted values and residuals
y_pred = X @ beta
residuals = y - y_pred
```

3. Evaluate the fitted line

For this eight-observation dataset:

$$\boxed{\widehat{y} = 1.2143 + 0.9524x}$$

$$\mathrm{SSE} = \sum_i (y_i - \widehat{y}_i)^2 \approx 3.9048$$

$$\mathrm{SST} = \sum_i (y_i - \bar{y})^2 = 42$$

$$\boxed{R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}} \approx 0.9070}$$

- SSE measures variation left unexplained by the line.
- R^2 is the fraction of sample variation explained by the fitted line.
- High training R^2 does not ensure good future predictions.

Code: SSE, SST, and R^2

```
sse = np.sum(residuals**2)
sst = np.sum((y - np.mean(y))**2)
r_squared = 1 - sse / sst

print("SSE:", sse)
print("SST:", sst)
print("R-squared:", r_squared)

# Pandas: display the results in a table
import pandas as pd
df = pd.DataFrame({"x": x, "y": y})

# Inspect observed values, predictions, residuals.
check = pd.DataFrame({
    "x": x, "observed": y,
    "predicted": y_pred, "residual": residuals
})
print(check.round(3))
```

4. Statistical inference: statsmodels

- Same OLS coefficient estimates as NumPy.
- Adds standard errors, t tests, p values, confidence intervals, and model diagnostics.
- Two interfaces: a design matrix or a formula similar to R.
- With only eight observations, interpret uncertainty carefully.

Code: statsmodels (matrix and formula)

```
# statsmodels: regression and statistical inference
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Matrix interface: X already has an intercept.
model_sm = sm.OLS(y, X).fit()
print(model_sm.summary())

# Formula interface: intercept added automatically.
model_formula = smf.ols("y ~ x", data=df).fit()
print(model_formula.summary())

# Equivalent R syntax:
# lm(y ~ x, data = df)
```

5. How certain is the estimated slope?

A slope of $\widehat{b}_1 = 0.9524$ is an *estimate* (another sample may yield another slope).

For the eight observations, $n = 8$ and $\mathrm{SSE} = 3.9048$:

$$\underbrace{s}_{\substack{\text{residual}\\ \text{standard error}}} = \sqrt{\frac{\mathrm{SSE}}{n-2}} = \sqrt{\frac{3.9048}{6}} \approx \boxed{0.8067}.$$

Since $\bar{x} = 4.5$ and $\sum_i (x_i - \bar{x})^2 = 42$,

$$\underbrace{\mathrm{SE}(\widehat{b}_1)}_{\text{slope standard error}} = \frac{s}{\sqrt{\sum_i (x_i - \bar{x})^2}} = \frac{0.8067}{\sqrt{42}} \approx \boxed{0.1245}.$$

- s estimates the standard deviation of the model errors.
- $\mathrm{SE}(\widehat{b}_1)$ measures uncertainty in the fitted slope.

5. How certain is the estimated slope? (continued)

Under the classical regression assumptions, a 95% confidence interval is

$$\widehat{b}_1 \pm t_{0.975,\, n-2}\, \mathrm{SE}(\widehat{b}_1).$$

With 6 degrees of freedom, $t_{0.975,6} \approx 2.447$:

$$0.9524 \pm 2.447(0.1245) \approx \boxed{[0.6478,\ 1.2570]}.$$

- The interval quantifies sampling uncertainty in the slope, not the variation of individual predictions.
- Assumptions include a linear conditional mean, independent errors, constant error variance, and normal errors for exact small-sample t inference.
- With only eight observations, diagnostics and interpretation matter.

Code: Slope uncertainty and 95% interval

```
# Residual standard error and slope standard error
n = len(x)
s = np.sqrt(sse / (n - 2))
se_slope = s / np.sqrt(np.sum((x - np.mean(x))**2))

print("Residual standard error:", s)
print("Slope standard error:", se_slope)

# Classical OLS inference (already fitted above)
print("Slope standard error:", model_sm.bse[1])
print("95% slope confidence interval:",
      model_sm.conf_int(alpha=0.05)[1])
```

6. Machine learning interface: scikit-learn

- Fit and predict using a standard estimator interface.
- Features must be a 2D matrix: $(n_samples, n_features)$.
- The score for `LinearRegression` is R^2.
- Here the score is evaluated *on training data*; testing on unseen data is a separate step.

Code: scikit-learn

```
# scikit-learn: fitting and prediction
from sklearn.linear_model import LinearRegression

# One feature, eight rows -> (8, 1)
X_sklearn = x.reshape(-1, 1)

lr_model = LinearRegression(fit_intercept=True)
lr_model.fit(X_sklearn, y)

print("Intercept:", lr_model.intercept_)
print("Slope:", lr_model.coef_[0])
print("Training R-squared:",
      lr_model.score(X_sklearn, y))

pred_sklearn = lr_model.predict(X_sklearn)
```

Code: Seaborn regression plot

```
# Seaborn and Matplotlib: visualize regression
import seaborn as sns
import matplotlib.pyplot as plt

# Scatterplot, OLS regression line, and a
# 95% confidence band for the mean response.
sns.lmplot(
    data=df,
    x="x",
    y="y",
    ci=95,
    height=5,
    aspect=1.2
)
plt.xlabel("Hours")
plt.ylabel("Output (hundreds of units)")
plt.show()
```

7. CVXPY: Regression as optimization

The least-squares objective can be written explicitly:

$$\boxed{\min_{\mathbf{b}} \ \|X\mathbf{b} - \mathbf{y}\|_2^2}$$

- **NumPy:** solve the least-squares system numerically.
- **CVXPY:** declare coefficients and an objective; ask a solver to minimize it.
- **PyTorch:** update parameters using calculated gradients.

Same fitted line, same squared-error objective,
different computational interfaces.

Code: CVXPY least squares

```
# CVXPY: declare and solve an optimization problem
import cvxpy as cp

# X and y come from the earlier NumPy slides.
beta_cvx = cp.Variable(2)
objective = cp.Minimize(
    cp.sum_squares(X @ beta_cvx - y)
)
problem = cp.Problem(objective)
problem.solve()

print("Intercept, slope:", beta_cvx.value)
print("SSE:", problem.value)
```

From Least Squares to Gradient Descent

All approaches minimize the same objective:

$$L(\beta) = \frac{1}{n}\|X\beta - y\|_2^2$$

NumPy	**CVXPY**	**PyTorch (SGD)**
Numerical least-squares solver	Convex optimization solver	Iterative gradient updates
Solve using linear algebra	Define objective and solve	Compute loss and gradients
No learning rate	No learning rate to specify	Choose learning rate α

Same objective, different computational methods.
Gradient descent approaches the least-squares solution when it converges.

8. From solver optimization to gradient descent

Same minimizing coefficients:

$$\underbrace{\sum_i (y_i - \widehat{y}_i)^2}_{\text{SSE}} \qquad \text{and} \qquad \underbrace{\frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2}_{\text{MSE}}.$$

Multiplying by $1/n$ does not change the minimizing parameters.

CVXPY solver	**PyTorch / SGD**
Declare an optimization problem	Define a model and loss
Call a solver	Choose a learning rate
Solver finds optimum numerically	Repeatedly update coefficients

Instead of delegating minimization to a solver, we now implement repeated gradient-based updates.

9. How gradient descent works

Write the prediction as $\widehat{y} = wx + b$ (weight w = slope; bias b = intercept).

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - wx_i - b)^2.$$

$$\frac{\partial L}{\partial w} = -\frac{2}{n} \sum_i x_i (y_i - wx_i - b), \qquad \frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i (y_i - wx_i - b).$$

$$\boxed{w \leftarrow w - \alpha \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}}$$

- Predict $\rightarrow$ measure error $\rightarrow$ compute gradients $\rightarrow$ update.
- Repeat until the changes in loss or coefficients are sufficiently small.

Code: PyTorch gradient descent (1/2)

```
# PyTorch: fit the line using gradient descent
import torch
import torch.nn as nn

torch.manual_seed(42)

# Tensor shapes: (8, 1)
X_tensor = torch.tensor(
    x.reshape(-1, 1), dtype=torch.float32
)
Y_tensor = torch.tensor(
    y.reshape(-1, 1), dtype=torch.float32
)

# One linear neuron: y_hat = weight*x + bias
linear_layer = nn.Linear(1, 1)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(
    linear_layer.parameters(), lr=0.01
)
```

Code: PyTorch gradient descent (2/2)

```
for epoch in range(2000):
    predictions = linear_layer(X_tensor)
    loss = criterion(predictions, Y_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

intercept_torch = linear_layer.bias.item()
slope_torch = linear_layer.weight.item()

print("Intercept:", intercept_torch)
print("Slope:", slope_torch)
print("Final MSE:", loss.item())
```

Code: Compare the fitted coefficients

```
results = pd.DataFrame({
    "Method": ["NumPy", "statsmodels", "Formula",
               "scikit-learn", "CVXPY", "PyTorch"],
    "Intercept": [beta[0], model_sm.params[0],
                  model_formula.params["Intercept"],
                  lr_model.intercept_, beta_cvx.value[0],
                  intercept_torch],
    "Slope": [beta[1], model_sm.params[1],
              model_formula.params["x"],
              lr_model.coef_[0], beta_cvx.value[1],
              slope_torch]
})
print(results.round(4))
```

10. Conclusions

- **NumPy:** solves a least-squares problem numerically.
- **statsmodels:** estimates coefficients and supports inference.
- **scikit-learn:** emphasizes fitting and prediction.
- **Seaborn:** shows the fitted relationship visually.
- **CVXPY:** formulates and solves an optimization objective.
- **PyTorch:** learns the same linear parameters via repeated gradient updates.

Why might an iterative method be useful when a problem becomes too large or complex for a direct solution?

Iterative methods, such as gradient descent, are useful because they avoid solving the entire problem at once.

For large or complex problems, direct solutions may require too much memory, computation, or may not exist in closed form.

Iterative methods instead improve the solution step by step, often using small batches of data. This makes them more scalable and flexible, especially for large datasets and neural networks.

Direct methods aim to compute a solution in one procedure; iterative methods gradually approach a solution through repeated updates.

11. Lab: Regression Diagnostics

1. **Question:** Identify the response, predictor, and units.
2. **Regression line:** Report the model; interpret the slope.
3. **Fit:** Compute predictions, residuals, SSE, residual MSE, and R^2.
4. **Diagnostics:** Plot the fitted line, residuals vs. fitted values, and a Q–Q plot; interpret the patterns.
5. **Uncertainty:** Report the slope's 95% confidence interval and relevant assumptions.
6. **Comparison:** Compare NumPy OLS with PyTorch gradient descent and CVXPY; explain differences.
7. **Conclusion:** What can eight observations support?

Submission

Submit one Jupyter notebook (.ipynb) with code, outputs, labeled plots, and short interpretations. Use the eight class observations.

Lab grading rubric

Task	Points
NumPy: OLS coefficients	1
Predictions, residuals, SSE, R^2	1
Statsmodels: inference and 95% CI	2
Regression and diagnostic plots	2
PyTorch: gradient descent comparison	2
CVXPY	1
Interpretation and conclusions	1
Total	**10**

Grading philosophy

Credit emphasizes genuine effort, documented progress, and demonstrated understanding rather than completing every task during class. Partial attempts can earn credit.

Lab expectations and discovery

- **Explore and discover.** Use the textbook, Python documentation, or internet resources to learn! For instance, how to create and interpret a Q–Q plot of *regression residuals*. Cite sources used.
- **Discuss and collaborate.** Help classmates at your table, but submit your own work; do not copy solutions.
- **Use AI responsibly.** It may support your learning, but must not complete the work for you. Understand any AI-assisted code.
- **Explain your decisions.** Be prepared to answer questions about your code, results, and reasoning.
- **Work during class.** Submit your progress by the end of the session, even if incomplete.

Remember

The goal is to learn through exploration—not simply reproduce code or obtain answers.